TECHLEARN ACADEMY

Conceitos de orientação a objetos

Curso

Programação Orientada a Objetos e UML

Módulo 1 - Classes, objetos e princípios da POO

Como estudar esta matéria

1. Leia a explicação teórica e acompanhe os exemplos do material.

2. Resolva os exercícios comentados antes de conferir suas justificativas. Anote suas dúvidas para conversar com o mentor.

3. Ao terminar os materiais do módulo, faça a avaliação de revisão disponível no site.

Origem do conteúdo

CQA/UNIP - Tecnologia em Análise e Desenvolvimento de Sistemas, Tomo 1. Páginas 6 a 9 do arquivo original.

O conteúdo da matéria começa na página 4 deste PDF. Antes dele, foram mantidas a capa e a página de créditos e avisos do documento original.

Os textos e exercícios originais conservam suas referências e o contexto de publicação. A organização em cursos e as avaliações de revisão pertencem ao projeto TechLearn Academy.

UNIP
UNIVERSIDADE PAULISTA

TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

MATERIAL INSTITUCIONAL ESPECÍFICO

TOMO 1

CQA - COMISSÃO DE QUALIFICAÇÃO E AVALIAÇÃO

TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

MATERIAL INSTRUCIONAL ESPECÍFICO

TOMO 1

Material instrucional específico, cujo conteúdo integral ou parcial não pode ser reproduzido ou utilizado sem autorização expressa, por escrito, da CQA/UNIP – Comissão de Qualificação e Avaliação da UNIP – UNIVERSIDADE PAULISTA.

Questão 2

Questão 2.[2]

Uma pizzaria fez uma ampliação de suas instalações e o gerente aproveitou para melhorar o sistema informatizado, que era limitado e não atendia a todas as funções necessárias. O gerente, então, contratou uma empresa para ampliar o software. No desenvolvimento do novo sistema, a empresa aproveitou partes do sistema antigo e estendeu os componentes de maneira a usar código validado, acrescentando as novas funções solicitadas.

Que conceito de orientação a objetos está descrito na situação hipotética acima?

A. Sobrecarga.

B. Herança.

C. Sobreposição.

D. Abstração.

E. Mensagem.

1. Introdução teórica

Sobrecarga, herança, hierarquia, sobreposição, abstração e mensagem

Sobrecarga é a provisão de mais de uma versão para um mesmo método. A diferenciação entre as versões é feita por assinaturas dos métodos, isto é, na lista de parâmetros que o método possui. Isso pode ser feito tanto na quantidade de parâmetros como no tipo de parâmetro informado (exemplos: *string, integer, double e float*). Os construtores também podem ser sobrecarregados.

Segue um exemplo de sobrecarga de construtor.

```
Point p1 = new Point(); // Construtor padrão
Point p2 = new Point(1,2); // Construtor sobrecarregado
```

Segue um exemplo de sobrecarga de método, alterando o tipo de dado.

```
int a = Math.abs(-10);
double b = Math.abs(-2.3);
```

[2]Questão 12 – Enade 2008.

Herança é o mecanismo utilizado em orientação a objeto para que uma classe (subclasse) herde as propriedades de outra classe (superclasse), isto é, seus atributos e seus métodos. Com a herança, podemos gerar uma hierarquia de classes na qual as no nível mais baixo herdam atributos e métodos da classe acima (suas características e seus comportamentos). A superclasse é a classe pai e a subclasse é a classe filha numa hierarquia de classes.

Segue um exemplo de herança.

Animal -> Mamífero -> Cachorro

O cachorro é um mamífero e também um animal. A classe Cachorro é uma subclasse de Mamífero. Mamífero é a superclasse de Cachorro. Cachorro é subclasse de Animal. Animal é superclasse tanto de Mamífero quanto de Cachorro.

A hierarquia é unidirecional, isto é, podemos falar que o cachorro é um mamífero, mas não podemos falar que o mamífero é um cachorro (um mamífero poderia ser uma baleia).

A sobreposição ocorre quando um método numa subclasse é declarado exatamente com o mesmo nome e lista de argumentos do método na superclasse, alterando ou não o código em seu interior.

Segue um exemplo de sobreposição.

```
class Mamifero {
    public void comer() {
     system.out.println("Mamifero comendo");
          }
}
class Cachorro extends Mamifero {
     public void comer() {
         system.out.println("Cachorro comendo");
         }
}
```

Abstração é a limitação de um amplo universo em um domínio específico. Com ela, focamos nos objetos, nas ações e nas prioridades que são relevantes para a aplicação específica, ignorando os demais pontos que são irrelevantes.

Segue um exemplo de abstração: em um domínio de determinado problema, podemos ter o conceito Veículo, que poderia significar grande quantidade de elementos (avião, carro, barco, submarino, disco voador, elevador, trem, metrô ou *jet-ski*). A abstração

reduz o universo ao qual se relaciona com a aplicação sendo desenvolvida. Por exemplo, em uma aplicação de Aluguel de Carros não usaríamos os conceitos de submarino e avião, dentre outros.

Mensagem é uma solicitação feita de um objeto para outro. Os objetos se comunicam por mensagens que geralmente são uma chamada de um método em algum dos objetos envolvidos no processo.

Segue um exemplo de mensagem: um objeto FormulárioCliente solicita uma consulta ao objeto PessoaFisica através do método ConsultarCliente(cpf).

2. Indicações bibliográficas

- GILLEANES, T. A. G. *UML 2 - uma abordagem prática*. São Paulo: Novatec, 2009.
- LARMAN, C. *Utilizando UML e padrões*. Porto Alegre: Bookman, 2007.
- SILVA, R. P. *Como modelar com UML 2*. Florianópolis: Visual Books, 2009.

3. Análise das alternativas

A – Alternativa incorreta.
JUSTIFICATIVA. A sobrecarga não permite o reaproveitamento ou a extensão de partes do sistema antigo, pois ela simplesmente gera novas versões dos métodos com assinaturas diferentes. Esses códigos terão de ser novamente testados e validados.

B – Alternativa correta.
JUSTIFICATIVA. A herança aproveita tudo que foi desenvolvido e aprovado na superclasse, possibilitando o uso nas subclasses como código já testado e validado.

C – Alternativa incorreta.
JUSTIFICATIVA. A sobreposição não aproveita partes antigas, mas as substitui. Esse novo código também terá de ser testado e validado.

D – Alternativa incorreta.
JUSTIFICATIVA. Abstração é um conceito que nada tem a ver com o reaproveitamento de código.

E – Alternativa incorreta.

JUSTIFICATIVA. A mensagem é um conceito que se refere à comunicação entre objetos, nada tendo a ver com o reaproveitamento de código em componentes já desenvolvidos.